

Rejuvenation Efforts in Python: A Qualitative Study of Type Hints Adoption

Oseias Magalhães
Computer Science Department,
University of Brasília
Brasília, Brazil
magalhaes.oseias@aluno.unb.br

Alana Mota
Computer Science Department,
University of Brasília
Brasília, Brazil
alana.mota@aluno.unb.br

Rodrigo Bonifácio
Informatics Center, Federal University
of Pernambuco
Brasília, Brazil
rbonifacio@cin.ufpe.br

Abstract

Type Hint modernization has become an important activity in the evolution of Python software as the language and its typing ecosystem continue to evolve. Although previous studies have characterized large-scale code transformations, little is known about the motivations that drive developers to perform these migrations, the practices they adopt, and how they perceive emerging AI-assisted development tools. To investigate these aspects, we conducted a mixed-method empirical study combining repository mining, developer surveys, thematic analysis, and qualitative triangulation.

Our findings indicate that large-scale Type Hint modernization is driven by the interplay between ecosystem evolution and software quality objectives. Although many modernization campaigns are initiated in response to changes in the Python ecosystem, developers frequently associate these transformations with expected benefits to code quality. Developers predominantly rely on deterministic automation tools, complemented by manual validation, whereas Foundation Models are viewed as useful for semantically complex tasks but are not yet considered suitable replacements for rule-based refactoring. Finally, triangulation between repository artifacts and developer feedback revealed strong semantic agreement, suggesting that repository artifacts provide a useful approximation of developers' motivations, while surveys offer complementary contextual insights.

Keywords

Keywords Software Rejuvenation, Python, Type Hints, Structural Pattern Matching, Repository Mining, Software Evolution, Refactoring, Developer Survey

1 Introduction

Python has become one of the most widely adopted programming languages due to its flexibility and developer productivity gains [6, 9, 17, 24, 26]. However, to remain competitive and suitable for modern software development, Python has continually evolved to incorporate new features. This continuous evolution requires existing code-bases to progressively rejuvenate in order to benefit from newer language capabilities and avoid software aging effects [15, 16, 19, 20]. These modernization efforts often involve repository-wide transformations to adopt newer typing syntax, improve compatibility with recent Python versions, simplify type annotations, and leverage advances in static analysis tooling.

To identify representative modernization efforts for qualitative investigation, we build upon a previous large-scale repository mining study that analyzed the evolution history of 141 popular open-source Python projects hosted on GitHub [25]. That study detected

more than 674,000 Type Hint-related source code transformations and identified commits containing extensive type-related modifications, providing a rich set of candidate cases for further investigation.

To better understand the motivations for those large-scale modernization efforts, we built a curated dataset of 117 representative large-scale commits from an initial set containing the 200 most important commits from each developer. The curation process prioritized commits that best exemplified substantial source code transformations and provided sufficient contextual information for qualitative investigation. A detailed description of this selection procedure is presented in 4. Each selected commit was authored by a distinct developer which was used for our subsequent qualitative analysis and developer survey.

Understanding these aspects is essential to complement mining-based studies with evidence that captures developers rationale and the context surrounding modernization efforts. Therefore, in this work, we extend the previous study by investigating the motivations and practices related to large-scale Python modernization.

For this end, we conducted a mixed-method empirical study combining a developer survey with qualitative analysis and triangulation. The survey focused on (i) motivations for performing large-scale Type Hints-related transformations, (ii) tools and automation techniques used during the process, and (iii) perceptions regarding the potential use of Foundation Models to support software modernization tasks. We analyzed the survey responses using thematic analysis and triangulated the resulting themes with repository artifacts, including commit messages, pull request discussions, and code review comments. This triangulation enabled us to assess the consistency between developer-reported motivations and the evidence observed in the repository history, providing a more comprehensive understanding of large-scale modernization practices.

Overall, this work bridges the gap between large-scale repository mining and a developer-centered understanding of software modernization. Some of the main contributions are:

- **A mixed-method investigation of large-scale Type Hint modernization.** We combine repository mining, developer surveys, thematic analysis, and qualitative triangulation to study the motivations, practices, and perceptions associated with large-scale modernization efforts.
- **An understanding of practices on Type Hints modernization.** We identify the technical and ecosystem-related motivations behind modernization campaigns and characterize the tools and practices developers adopt to perform them.

- **Insights into the emerging role of Foundation Models in software modernization.** We characterize developers' current perceptions of AI-assisted modernization, highlighting both growing acceptance and continued preference for deterministic tooling in rule-based transformations.
- **Evidence supporting the use of repository artifacts as a source of developer rationale.** Through qualitative triangulation, we show that repository artifacts and developer reports are largely consistent, while surveys contribute additional contextual explanations that are often absent from project history.

The remainder of this paper is organized as follows. Section 2 provides the background on Python language features and source-code transformations. Section 3 reviews the related work. Section 4 describes the research approach, including the study design, data collection, and analysis procedures. Section 5 presents the results of the empirical study. Section 6 discusses the main findings and their implications. Section 7 outlines the threats to the validity of the study. Finally, Section 8 concludes the paper and discusses directions for future work.

2 Background

Python has continuously evolved by introducing language features intended to improve software maintainability and supporting large-scale software development. Among these, Type Hints were introduced to improve code readability, standardize the specification of type information, and enable the detection of type-related issues through integration with static analysis tools. In our previous work, we quantitatively investigated large-scale Type Hints adoption by mining software repositories and identifying commits involving extensive type-related source code transformations. Although that study characterized how these transformations occur in practice, it did not investigate the motivations, decision-making processes, or development practices underlying them. This paper extends that work through a qualitative, developer-centered investigation based on survey responses and artifact triangulation.

2.1 Type Hints

Type Hints were standardized in PEP 484 [12] as part of Python's gradual typing system to address the lack of a standardized mechanism for expressing type information in Python programs. Prior to their adoption, developers relied on heterogeneous approaches, including decorators, docstrings, and conventions implemented by third-party libraries, resulting in incompatible annotation styles and limited interoperability among tools [11]. This lack of standardization also hindered IDE support and automated source code transformation capabilities.

PEP 484 established a common vocabulary for annotating function parameters, return types, and variables, enabling external tools to perform offline static analysis while preserving Python's dynamically typed nature [12]. Subsequent proposals, including PEP 526 [13] and PEP 585 [14], extended typing support by introducing native variable annotations and built-in generic types. In particular, PEP 526 replaced comment-based annotations (e.g., `# type:`

annotation), which suffered from readability limitations, artificial variable initializations, and poor accessibility for automated tools [13].

Unlike statically typed languages, Python does not use type annotations during runtime. Instead, they provide semantic information for external tools such as MyPy and Pyright, which exploit them for offline static analysis, code completion, navigation, and automated source code transformations [12]. Previous work also reports benefits including earlier bug detection, improved software quality, and better software maintenance activities [23]. Importantly, Type Hints remain entirely optional, preserving Python's dynamically typed execution model while enabling optional analysis and validation by external tools and libraries [12, 13], as illustrated in Figure 1.

```
def greeting(name: str) -> str:
    return 'Hello ' + name
```

Listing 1: Example of a function with Type Hints in Python (PEP 484 [12]).

2.2 Program Transformations

In this study, we investigate source code transformations involving Python type annotations. These transformations include the introduction, removal, and modification of type annotations in variables, function parameters, and return types, reflecting how developers evolve typing information as projects adopt modern Python features. By analyzing these transformations, we investigate the motivations, practices, and decision-making processes underlying large-scale software modernization.

2.3 Large-Scale Modernization

Large-scale modernization refers to systematic source code transformations that affect many locations across a codebase within a single development effort. Such transformations are commonly associated with software reengineering and automated program transformation, particularly when their scale makes manual application impractical[1]. These efforts may be motivated by the adoption of newer language features, improvements in code quality, and keep projects aligned with the evolution of the programming language. Figure 1 illustrates a representative example from the aiohttp project (commit 35754d4), where type annotations were consistently updated across the repository by replacing legacy typing constructs with their modern counterparts, such as migrating generic types introduced by PEP 585 [14], simplifying optional types using PEP 604 [22], and updating imports according to current Python recommendations. This example highlights how software modernization is often performed through coordinated, repository-wide transformations.

```

--- a/aiohttp/_cookie_helpers.py      (Python 3.8 style)
+++ b/aiohttp/_cookie_helpers.py      (Python 3.9+ style)
@@ -6,6 +6,7 @@
import re
+from collections.abc import Sequence
from http.cookies import Morsel
-from typing import List, Optional, Sequence, Tuple, cast
+from typing import cast

@@ -159,9 +160,9 @@
def parse_cookie_header(header: str) ->
- List[Tuple[str, Morsel[str]]]:
+ list[tuple[str, Morsel[str]]]:
...
- cookies: List[Tuple[str, Morsel[str]]] = []
+ cookies: list[tuple[str, Morsel[str]]] = []
...
- current_morsel: Optional[Morsel[str]] = None
+ current_morsel: Morsel[str] | None = None

```

Figure 1: Python type annotation modernization in aiohttp (commit 35754d4).

2.4 Foundation Models

Foundation models are large-scale machine learning models trained on broad and diverse datasets and designed to support a wide range of downstream tasks through adaptation or direct use [10]. Unlike models developed for a single predefined task, foundation models provide general-purpose capabilities that can be reused across different application domains. Recent advances in the scale of training data, model architectures, and computational resources have enabled foundation models to perform increasingly diverse tasks involving natural language, code, images, and other modalities.

In software development, foundation models trained on source code and natural language have enabled new forms of programming assistance, including code generation, code completion, explanation, summarization, and source code transformation. Large language models (LLMs) are a prominent class of foundation models that can process and generate natural language and programming languages, allowing developers to interact with them using natural-language instructions. These capabilities have introduced new possibilities for automating or assisting software engineering activities, while still requiring developers to validate and integrate the generated results into existing development workflows.

3 Related Work

Software rejuvenation refers to the process of updating legacy codebases to incorporate modern programming language features while preserving their original semantics [21]. Unlike general refactoring, which focuses on improving internal structure, rejuvenation addresses technical obsolescence driven by language evolution and ecosystem changes. Prior work has highlighted that such modernization efforts often involve trade-offs between adopting new language features and maintaining backward compatibility in existing systems [7].

Empirical studies on Python evolution have primarily focused on the adoption of specific language features. Research on type hints shows that their adoption initially emerged at low rates and is

typically partial, often concentrated on function signatures rather than full codebases [8, 23]. Once introduced, type annotations tend to remain stable over time, with adoption largely motivated by improved documentation and tool support such as static analysis and IDE assistance [2].

While existing studies have primarily focused on adoption rates, automated transformations, or language design, fewer works investigate the motivations behind large-scale code modernization from the developer’s perspective. Some recent efforts have begun to incorporate qualitative analysis of development artifacts to better understand intent behind changes [5]. However, these studies still rarely combine repository mining with direct developer feedback.

In contrast, this study investigates the motivations behind software rejuvenation in Python systems by combining repository mining with thematic analysis and triangulation of commit messages, pull requests, and developer responses. This enables a complementary perspective that connects observable code changes with the underlying intentions expressed by developers.

4 Approach

This study investigates the motivations and practices associated with large-scale Python modernization efforts involving Type Hints. Figure 2 presents an overview of the research workflow. While our previous research (based on a large-scale repository-mining analysis) characterizes how these transformations occur, it does not reveal the reasoning behind developers’ decisions, the role of automation in modernization, or developers’ perspectives on emerging AI-based support. To address these aspects, we conducted a survey with authors of commits containing a large number of detected type-related transformations.

Specifically, we aim to understand why developers introduce and evolve type annotations, whether they rely on tools to support such modifications, and how they perceive the potential use of Foundation Models for similar tasks. To guide this investigation, we address the following research questions:

- **RQ1: What motivations drive developers to perform large-scale Type Hint modernization efforts?** This question investigates the related factors that motivate developers to modernize Type Hints across large open-source Python projects.
- **RQ2: What tools or automation techniques do developers use to support large-scale Type Hint modifications?** This question investigates the tools, scripts, and automation strategies developers employ to facilitate large-scale Type Hint modernization.
- **RQ3: How do developers perceive the potential use of Foundation Models to assist Type Hint modernization tasks?** This question seeks to identify developers’ expectations, perceived benefits, concerns, and potential use cases for applying to Type Hint modernization.

We analyze survey responses using thematic analysis and integrate these findings with evidence extracted from commits and pull request discussions. We then perform a triangulation of evidence across the three sources (commits, pull requests, and survey responses), assessing consistency between observed code changes and developers’ reported motivations and practices.

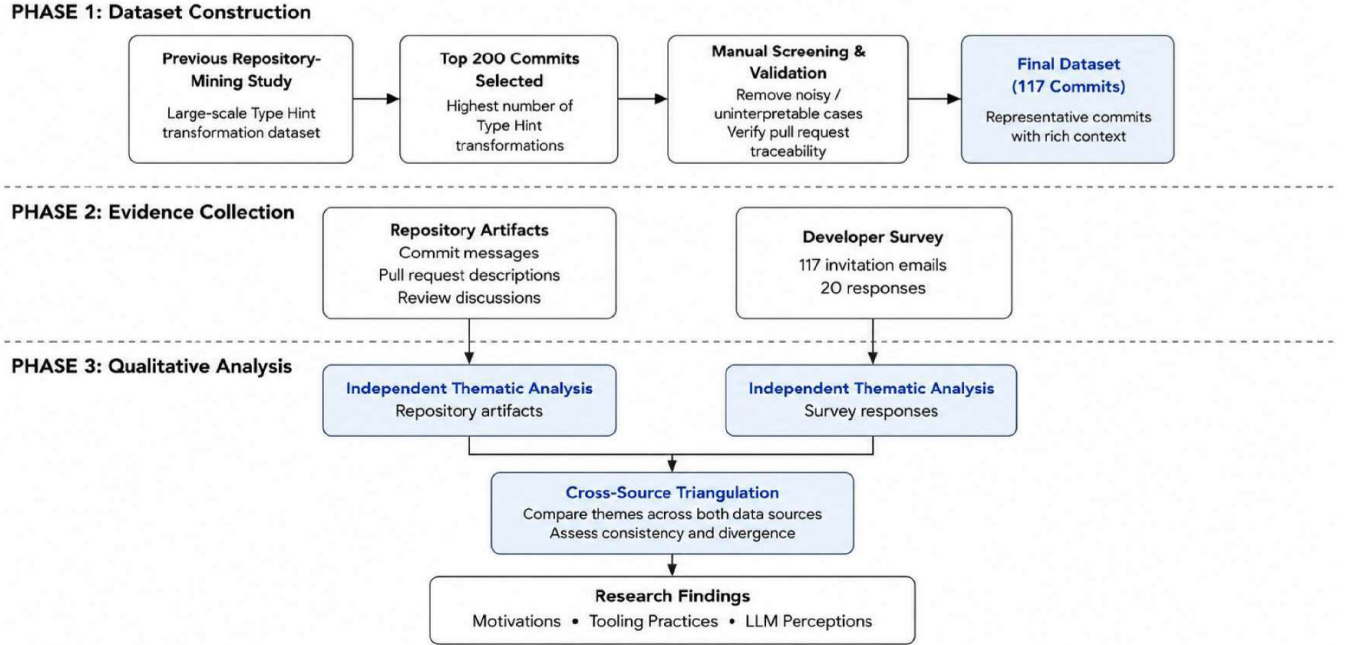


Figure 2: Overview of the research methodology.

This triangulation enables a more comprehensive understanding of large-scale Type Hints modernization by connecting observable software evolution with developer intent and decision-making processes.

4.1 Study Settings

We build upon a large-scale mining dataset of 141 popular Python repositories, selecting the top 200 commits with the highest number of Type Hints-related transformations. From this set, we manually curate a final dataset of 57 projects and 117 representative commits based on their relevance and the richness of contextual information available in the associated pull requests.

For each selected commit, we retrieve and analyze the corresponding pull request discussions when available. In cases where no pull request was explicitly linked, we manually verify the repository history to ensure traceability. Each commit is associated with a single primary pull request, and this relationship was validated manually.

To complement repository analysis, we contacted the authors of the 117 selected commits via email. Although 10 developers

were unreachable, we obtained 20 valid responses from 20 different projects. The survey is exploratory in nature and aims to capture developers' perspectives on motivations, tooling practices, and the potential use of Foundation Models in modernization tasks.

4.2 Data Collection

4.2.1 Commit Selection. Starting from the dataset produced in our previous study, we identified candidate commits according to two criteria: (i) the number of detected type-related source code transformations and (ii) recency, considering commits authored between 2024 and 2026. This subset accounts for 39.43% of all detected transformations. To avoid over representing highly active contributors, we retained only the commit containing the largest number of detected transformations for each developer, ensuring that each developer was represented only once. These commits represent 24.28% of transformations within the selected time period. We then excluded commits associated with GitHub *noreply* accounts, since their authors could not be reliably contacted for the survey. The remaining commits were ranked according to their number of detected transformations then we selected the top 200,

this subset covers 58.37% of all transformations in the developers’ largest commits during the selected period. Providing a manageable yet highly representative sample for the subsequent qualitative analysis.

4.2.2 Survey Procedure. We manually inspected each of the selected commits to assess whether the detected transformations were sufficiently evident and interpretable for a follow-up developer survey. Commits containing numerous unrelated modifications, where the detected transformations were part of broader code changes and their rationale could not be reliably inferred, were excluded. We also discarded cases lacking sufficient contextual information (e.g., commit messages or associated pull requests discussions) to support the qualitative analysis and survey design. This manual screening resulted in a final dataset of 117 commits.

We contacted the authors of the 117 selected commits through an e-mail survey containing the questions presented in Table 1. The questionnaire investigated (i) the motivations behind the transformations, (ii) the tools and automation practices adopted during the modernization process, and (iii) developers’ perceptions regarding the use of language models for similar tasks.

Table 1: Script of the survey questions

RQ	Question
RQ1	What was your decision-making process and rationale for introducing this change?
RQ2	Did you use any tools or automation to support this large-scale modification?
RQ3	Do you envision using language models to assist with this kind of task?

Overall, we received 20 responses (17.09% response rate), each from different open-source Python projects. The respondents include developers from widely used projects such as *FastAPI*, *aiohttp*, *Scrapy*, *MyPy*, *Pipenv*, *Locust* and *Chia Blockchain*, including spanning domains. This diversity widens the range of perspectives captured.

4.2.3 Repository Artifact Collection. For each of the 20 survey responses, we collected the corresponding repository artifacts to support a triangulated analysis of software development decisions. Each case was anchored on a surveyed commit and expanded using the GitHub API to retrieve all associated contextual artifacts.

Specifically, we retrieved commit messages, pull request descriptions, and review-related discussions. Pull request data was obtained using the `/commits/sha/pulls` endpoint, while additional discussion context was collected using the issues and pull request review APIs, including issue comments (`/issues/number/comments`), review comments (`/pulls/number/comments`), and structured review events (`/pulls/number/reviews`). Consequently, each case in the dataset consists of multiple complementary sources of evidence: the developer’s survey response, the associated commit message, the pull request description, and the corresponding review and discussion artifacts.

To construct each triangulated case, all artifacts were grouped by their associated commit and pull request identifiers, ensuring that

all related discussion and review data were aggregated into a single analytical unit. This allowed us to capture both implementation-level decisions and the surrounding design rationale expressed during the review process.

To support qualitative analysis, the first two authors independently recorded observations for each case. For each artifact type, separate notes were produced summarizing the technical motivations, design considerations, and decision-making rationale expressed by developers across commits, pull requests, and review discussions.

4.3 Data Analysis

To investigate developers’ decision-making processes behind large-scale code changes, the use of tools and automation during these modifications, and their perceptions regarding the use of large language models (LLMs) to support such tasks, we analyzed 20 survey responses using thematic analysis following Braun and Clarke [4].

The unit of analysis consisted of two complementary data sources: (i) textual excerpts extracted from commits, pull request descriptions, and code review comments, and (ii) developers’ survey responses. Both sources were analyzed separately using thematic analysis. These excerpts were manually coded at a fine-grained level using a multi-label coding scheme, in which multiple codes could be assigned to the same segment when appropriate. The coding was performed explicitly, without inferring categories not directly supported by the text. Although most codes emerged from the data, some initial categories were informed by prior literature and were subsequently refined throughout the analysis process. Excerpts considered irrelevant according to predefined criteria—such as purely social interactions or comments without technical implications—were excluded from coding.

Following Braun and Clarke’s guidelines, the analysis proceeded through iterative cycles of data familiarization, code generation, grouping related codes into candidate themes, and continuous refinement until a stable thematic structure was achieved. Rather than following a strictly linear pipeline, the coding and theme refinement process was revisited multiple times to ensure consistency and conceptual coherence. The process was conducted in pairs, with independent coding by two researchers followed by comparison and reconciliation of differences. This cross-validation process helped increase consistency and reduce individual interpretation bias.

We also followed established practices for trustworthiness in qualitative research, including triangulation of interpretations between researchers, discussion meetings among researchers (peer debriefing), debate of individual perspectives during the analysis, investigator triangulation, and cross-checking of coding decisions until reaching consensus among the researchers [18].

We used ATLAS.ti [3] to support data organization, coding management, and traceability of analytical decisions, enabling an audit trail of code evolution and theme refinement. This contributed to the transparency and reliability of the analytical process.

Overall, the resulting themes characterizing developers’ motivations, modernization practices, and perceptions of AI-assisted transformations are presented in Section 5.

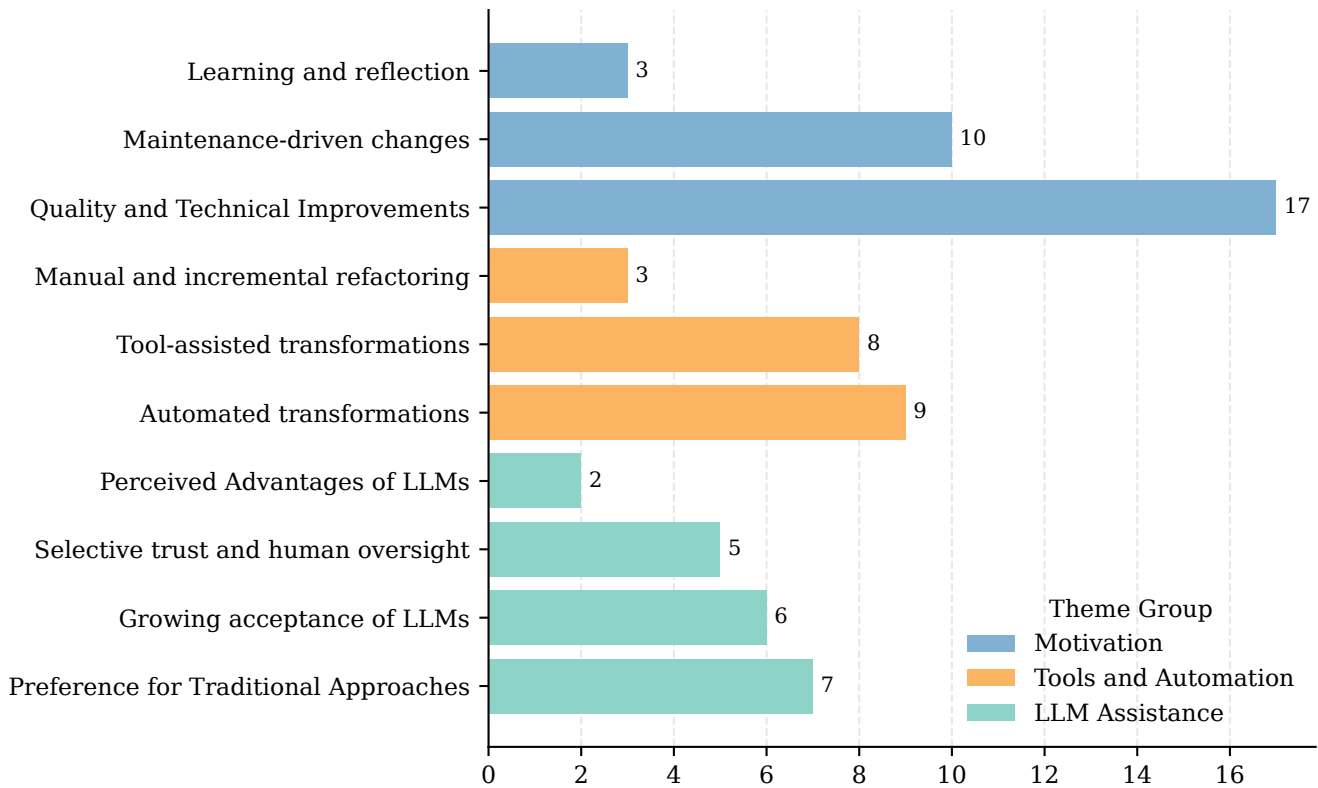


Figure 3: Themes identified from developers' feedback across the three research questions. The numbers represent the frequency of quotations assigned to each theme.

4.4 Triangulation

For each of the 20 cases, we performed a qualitative triangulation of developer intent by comparing the results of two thematic analyses: one conducted on repository artifacts (commit messages, pull request descriptions, and code review comments) and another conducted on survey responses. Each case corresponded to a single commit, its associated pull request and review discussion, and the corresponding developer survey response, treated as a unified unit of analysis.

The themes identified through thematic analysis were compared across these sources without re-coding the data, focusing on whether the same underlying motivation was expressed across different artifacts. This comparison assessed the degree of consistency between developers' reported motivations and the motivations observed in repository artifacts.

Consistency was evaluated qualitatively based on the extent to which motivations converged across sources, considering both alignment and partial divergence in emphasis or detail.

This process was conducted independently by two researchers, and disagreements in interpretation were resolved through discussion until consensus was reached.

5 Results

5.1 Overview of Developer Survey Results

We contacted developers responsible for large-scale modernization commits and then manually validating the candidate commits, we sent 117 emails to developers associated with the recent transformations (2024-2026), while 10 developers were not reachable. Resulting in 107 developers successfully contacted and 20 responses received.

The reported motivations largely confirm the patterns observed in the mining study. Most developers described the changes as modernization or maintenance activities driven by the evolution of the Python ecosystem, adoption of newer language features, improved readability, better documentation, and stronger static analysis support (FASTAPI project – commit 3da206c, AIOHTTP project – commit 35754d4, WAGTAIL project – commit 80b1ebe)

This is reflected in statements such as:

“Better readability and more expressive code was the main motivation for this change.” (WAGTAIL project, commit 80b1ebe)

and

“The motivation was due to the deprecation of Python 3.9.” (FASTAPI project, commit 3da206c)

Several respondents also highlighted defect prevention and long-term maintainability as important benefits of Type Hints (CHIA

BLOCKCHAIN project, commit a447734, DJANGO-CMS project, commit 1efaa98, AUTOGPT project, commit e16e69c). For instance:

“This makes the code more resilient against bugs.” (CHIA BLOCKCHAIN project, commit a447734)

Automation played a central role in many modernization efforts. Developers frequently reported using tools such as `pyupgrade`, `ruff`, `mypy`, and IDE-assisted search-and-replace mechanisms to perform repetitive changes (TORNADO project, commit 113cf79, PYTORCH VISION project, commit a095de1, MYPY project, commit 6682563).

This is supported by statements such as:

“This commit was generated as part of automated transformations using pyupgrade.” (TORNADO project, commit 113cf79)

However, nearly all respondents emphasized the need for manual inspection after automated transformations, particularly in large-scale commits, due to concerns about incorrect modifications and unintended side effects (CVAT project, commit 5ddeda3, FREQTRADE project, commit f369151).

Opinions regarding Foundation Models were more diverse. Some developers viewed LLMs as potentially useful for repetitive typing-related tasks and reported positive experiences using them for code transformations (DEEP AGENTS project, commit d398a1e, PIPECAT project, commit c4dbe92). For example:

“Virtually everyone is using agents / LLMs these days for coding.” (DEEP AGENTS project, commit d398a1e)

and

“Claude Code did almost everything right, but I had to manually edit a few things.” (PIPECAT project, commit c4dbe92)

Others expressed concerns about reliability, predictability, and loss of control over generated changes, preferring deterministic tools for large-scale refactoring activities (WAGTAIL project, commit 80b1ebe, LOCUST project, commit 9341cf9, SCRAPY project, commit 6ecc9e0).

Finally, the responses are consistent with an important finding from our quantitative analysis: although many Type Hint transformations are mechanical and highly automatable, successful modernization still depends on developer supervision and project-specific decisions (MANIM project, commit 8773252).

5.2 Thematic Analysis of Developer Responses

5.2.1 Motivations for Code Changes. The thematic analysis revealed that large-scale type-related transformations were primarily motivated by quality and technical improvements ($n = 17$). Developers frequently associated these changes with the adoption of modern Python practices, improved readability and maintainability, better IDE support, enhanced static analysis capabilities, and the prevention of subtle bugs. Other responses emphasized architectural concerns, such as reducing code duplication, introducing type abstractions, and addressing limitations in legacy designs. For example, a developer from the WAGTAIL project (commit 80b1ebe) explained that adopting newer language features:

“would make the code more readable and expressive” (WAGTAIL project, commit 80b1ebe).

while a contributor to (CHIA BLOCKCHAIN project, commit a447734) noted that the changes were intended to make the code:

“more resilient against bugs” (CHIA BLOCKCHAIN, commit a447734).

Maintenance-driven changes represented the second most common motivation theme ($n = 10$). These responses were predominantly associated with the evolution of the Python ecosystem, including the discontinuation of support for older Python versions and dependency maintenance requirements. One respondent from the PYTORCH VISION project (commit a095de1) explicitly stated that the change was motivated by:

“the motivation was due to the deprecation of Python 3.9.” (PYTORCH VISION project, commit a095de1)

whereas another contributor described the modifications as routine maintenance activities. In some cases, ecosystem pressures and quality concerns appeared simultaneously. For instance, a developer from the WAGTAIL project (commit 80b1ebe) reported that support for Python 3.8 was being dropped due to its end-of-life status, while newer language features were adopted to improve code quality.

Finally, a small number of responses highlighted learning and reflection as motivations for modernization efforts ($n = 3$). Developers described these transformations as opportunities to gain familiarity with the codebase, improve their understanding of Python typing, and reflect on previous design decisions. A developer from the MANIM project (commit 8773252) noted that:

“Adding type annotations allowed me to learn about the source code... improve my skills related type annotations in python.” (MANIM project, commit 8773252).

Overall, these findings suggest that large-scale modernization efforts are driven by a combination of internal quality concerns and external ecosystem pressures, while compatibility and maintenance requirements often trigger modernization efforts, developers also perceive these changes as opportunities to improve code quality, simplify architectures, and strengthen the long-term maintainability of their projects.

5.2.2 Refactoring Tools and Practices. Regarding the transformation practices employed, we observe a strong reliance on automation for large-scale refactorings ($n = 9$). Developers described the use of deterministic tools and scripts, including `pyupgrade`, `ruff`, `sed`, and custom automation scripts, especially in version migration and syntactic modernization tasks. These tools enabled deterministic and repeatable transformations across multiple files simultaneously. (e.g., TORNADO project, commit 113cf79, FASTAPI project, commit 3da206c, WAGTAIL project, commit 80b1ebe, AIO-HTTP project, commit 35754d4, PYTORCH VISION project, commit a095de1, PIPENV project, commit d6dc5a0).

“This commit was part of a series of pyupgrade-driven changes... I’d rather use a deterministic tool than an LLM.” (TORNADO project, commit 113cf79)

Tool-assisted transformations were also commonly reported ($n = 8$), where developers relied on conventional development tools, such as IDE functionalities, search-and-replace operations, reference updates and utilities such as `grep`. These tools supported developers carrying out and validating modifications. (e.g., AUTO-GPT project, commit e16e69c, CHIA BLOCKCHAIN project, commit

a447734, CVAT project, commit 5ddeda3, ROBOFLOW SUPERVISION project, commit fd1277f, MYPY project, commit 6682563).

“I probably used VSCode’s find/replace all feature, but no AI tools in this instance.” (AUTOGPT project, commit e16e69c)

Finally, a smaller number of responses ($n = 3$) emphasized incremental refactoring practices. Developers described performing gradual and manual transformations with no tool support. This approach was commonly adopted to cope with architectural constraints and to reduce the risks associated with large-scale modifications. (e.g., MANIM project, commit 8773252, SCRAPY project, commit 6ecc9e0, LOCUST project, commit 9341cf9).

“I go file by file, fixing a few type errors at a time and re-running the type checker repeatedly until convergence.” (MANIM project, commit 8773252)

5.2.3 Perceptions of Large Language Model Usage. With respect to the use of foundation models, the responses reveal a predominance of skeptical or restrictive views regarding their application to simple or deterministic refactoring tasks ($n = 7$). This is reflected in statements such as:

“As long as pyupgrade is maintained, I’d rather use a deterministic tool than an LLM.” (TORNADO project, commit 113cf79).

“That’s not something we need LLMs for - it’s a straight-forward rule-based replacement.” (FREQTRADE project, commit f369151).

These findings suggest that developers often view type-related transformations as tasks that are already well supported by established tooling ecosystems.

Despite this preference for conventional approaches, several responses reflected a growing acceptance of LLM-assisted development ($n = 6$). Some developers reported having successfully employed tools such as Claude Code for modernization tasks, whereas others anticipated relying more heavily on language models or coding agents in future refactoring activities. One participant remarked that:

“virtually everyone is using agents / LLMs these days for coding” (DEEP AGENTS PROJECT, commit d398a1e).

While another stated that, if performing the same transformation today, they would

“100% use a language model” (CHIA BLOCKCHAIN PROJECT, commit a447734).

A set of responses emphasized selective trust and human oversight ($n = 5$), including the occasional use of tools such as Claude Code. In these cases, developers described using LLMs primarily for consultation or guidance rather than directly incorporating generated code into pull requests. Even when positive experiences were reported, respondents highlighted the need for manual inspection and correction. For instance, one developer noted that

“Claude Code did almost everything right, but I had to manually edit a few things” (PIPECAT, commit c4dbe92).

illustrating a cautious approach toward AI-assisted development.

Finally, a few responses highlighted specific advantages of LLMs ($n = 2$), particularly for transformations involving semantic understanding rather than purely mechanical replacements. In such cases, developers considered LLMs more convenient than traditional tools and envisioned their application to broader migration tasks where deterministic rules are insufficient.

Overall, the findings suggest that the adoption of LLM for large-scale type-related refactorings is not determined by awareness of the technology itself, but rather by concerns related to predictability, verification overhead, and integration with established deterministic toolchains. Consequently, LLMs are currently positioned as a potential complementary tool to support development workflows, but not replace existing rule-based refactoring pipelines.

5.3 Thematic Analysis of Repository Artifacts

We also conduct a thematic analysis of repository artifacts, such as commit messages and pull request discussions related to the 20 survey feedback received. The results reveals that repository artifacts primarily emphasize three complementary aspects of large-scale type hint modernization: the motivations that trigger migration efforts, the technical outcomes produced by these changes, and the strategies adopted to perform them, as shown in Figure 4.

Regarding motivations, most evidence falls under *Modernization Infrastructure* ($n = 10$). Repository discussions frequently associate large-scale transformations with broader maintenance initiatives, such as dropping support for legacy Python versions, adopting newer language features, enabling static analysis tools or aligning the codebase with updated development practices. Rather than presenting type annotations as isolated improvements, developers often describe them as part of larger modernization campaigns affecting the entire project infrastructure.

The analysis of outcomes shows that repository discussions primarily focus on the resulting evolution of the type system, with *Type System Evolution* accounting for 19 quotations. Pull request conversations frequently discuss the technical consequences of the migration, including improvements in type inference, static checking, compatibility with modern typing syntax, and the resolution of typing inconsistencies. This emphasis is reflected in comments such as:

“This is mostly typing changes, to use built-in functionality instead of imports from the typing module.” (TORNADO project, PR #3596)

This predominance reflects the collaborative nature of pull request reviews, where developers spend more effort discussing the correctness and implications of typing decisions than the motivations behind the migration itself.

Finally, the identified strategies indicate that modernization efforts are rarely performed through isolated edits. Most quotations describe coordinated migration strategies, again concentrated in *Modernization Infrastructure* ($n = 14$), such as repository-wide automated rewrites, the adoption of tools like pyupgrade, systematic migration to newer typing syntax, and project-wide cleanup after dropping support for older Python versions. One representative example explicitly describes this workflow:

“Update type annotations with py3.8 features. Automated change with pyupgrade (restricted to the typing

plugins), followed by manual removal of unused imports.” (TORNADO project, commit 113cf79)

Additional strategies involve code restructuring and API redesign ($n = 6$), suggesting that type hint modernization is frequently performed alongside broader refactoring activities.

Overall, these findings suggest that large-scale type hint transformations are embedded within broader software evolution initiatives. While the initial motivation is commonly driven by infrastructure modernization and ecosystem evolution, repository discussions predominantly concentrate on ensuring the technical correctness of the resulting type system. This observation complements the evidence obtained from developers’ responses by showing more technical discussions about how modernization migrations are implemented and the collaborative review process.

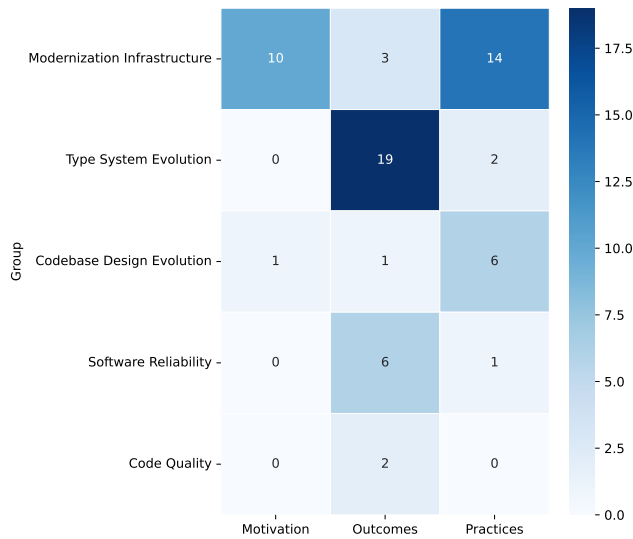


Figure 4: Themes identified from repository artifacts analysis.

5.4 Triangulation Results

To assess the consistency between available repository artifacts and developers’ feedback, we compared the themes identified from commit messages, pull request discussions, and code review comments with those obtained from the survey responses. Overall, the triangulation revealed a high level of agreement between both sources. 85% of the analyzed cases demonstrated at least partial convergence regarding the motivations behind the modernization effort.

Beyond validating the consistency of the findings, the triangulation also showed that the two sources provide complementary perspectives on the same development activity, as shown in Figure 5. Repository artifacts primarily document the technical rationale of the changes, emphasizing modernization campaigns, Python version migrations, tooling adoption, and implementation strategies. In contrast, survey responses more frequently expose the developers’ decision-making process, highlighting perceived benefits such as improved readability, maintainability, type safety and code

quality. Consequently, both sources differ in the level of abstraction and the aspects they emphasize.

Across the cases, the most recurrent motivation categories were Typing Improvements ($n = 11$), Maintainability and Refactoring ($n = 9$), Language Evolution and Modernization ($n = 8$), and Compatibility ($n = 6$). Less frequent themes included Correctness and Reliability ($n = 3$), Tooling Support ($n = 3$), Standardization ($n = 1$), Codebase Understanding ($n = 1$), and one case classified as Undetermined. These categories closely match the dominant themes independently identified in both thematic analyses, reinforcing that large-scale type hint modernization is primarily motivated by software quality improvements, adoption of evolving Python language features, and long-term maintenance concerns rather than isolated code changes.

The observed convergence increases confidence that repository artifacts provide a reliable approximation of developers’ motivations in large-scale modernization efforts. At the same time, the survey responses reveal contextual information that is rarely documented in commits and pull request discussions, suggesting that combining both sources produces a richer understanding of developer intent than relying on either source alone.

6 Discussion

This section discusses the implications of our findings in light of the research questions. We interpret the evidence obtained from the developer survey, repository artifacts, and their triangulation to better understand the motivations of developers performing large-scale Type Hint modernization, the practices adopted that support large-scale transformations and the developers perception of Foundation Models in supporting such activities.

6.1 RQ1: What motivations drive developers to perform large-scale Type Hint modernization efforts?

Our findings suggest that large-scale Type Hint modernization is driven by the interplay between ecosystem evolution and software quality objectives. Although many modernization campaigns are initiated in response to changes in the Python ecosystem, developers consistently associate these transformations with expected benefits to code quality, including improving maintainability, readability, static analysis support, and reducing the likelihood of defects.

This interpretation is reinforced by the convergence of evidence across repository artifacts and developer feedback. Commit messages and pull request descriptions frequently emphasize the technical aspects of the modernization process, such as migrating to newer typing syntax, updating compatibility with recent Python versions and tooling adoption. In contrast, developers responses reveal the broader motivations underlying these changes related to quality and maintainability.

These findings complement previous repository-mining studies by providing insight into the motivations behind large-scale modernization efforts. Our results suggest that Type Hint modernization is perceived as part of long-term software evolution, where ecosystem changes act as triggers for improvements, while developers leverage these opportunities to introduce improvements that strengthen software evolution.

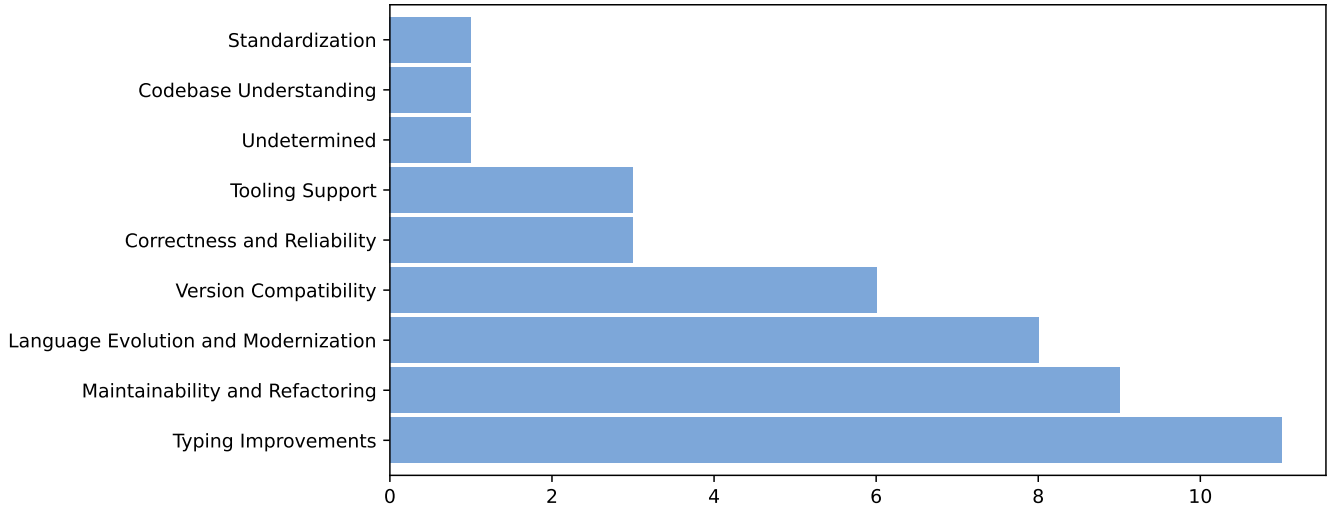


Figure 5: Frequency of motivation categories identified across the triangulated cases.

6.2 RQ2: What tools or automation techniques do developers use to support large-scale Type Hint modifications?

Our results reveal that deterministic automation tools are broadly used for executing large-scale modernization. Tools such as `pyupgrade`, `ruff`, IDE-assisted refactorings, and custom scripts enable developers to perform thousands of type-related transformations.

However, the usage of deterministic automation tools is not perceived as a fully autonomous process. We also note a concern with ensuring that large-scale transformations are verified, as well as a focus on deterministic tools that guarantee deterministic behavior. Automation is consistently accompanied by manual verification. Rather than delegating the entire modernization process to tools, developers described workflows in which automated transformations generate candidate modifications that are subsequently reviewed and validated before integration.

This observation helps explain how developers performing large-scale type-related transformations. These modernizations appear to depend less on manually individual editing than on selecting reliable transformations tools and ensuring that works correctly, where deterministic automation performs repetitive transformations while developers remain responsible for validating semantic correctness and project-specific decisions.

6.3 RQ3: How do developers perceive the potential use of Foundation Models to assist Type Hint modernization tasks?

Our results suggest a growing openness among developers to incorporate Foundation Models into their modernization workflows, but not yet as replacements for deterministic transformation tools. While several participants reported positive experiences using LLMs for programming tasks and anticipated an increasing role

for AI-assisted development, they also expressed reservations regarding the reliability, predictability, and verification of large-scale automated changes.

There is a temporal aspect that helps explain the coexistence of seemingly contrasting themes identified in the analysis. While deterministic tools remain the preferred choice for large-scale mechanical transformations, developers also reported growing confidence in Foundation Models and increasingly viewed them as viable assistants for future modernization efforts. Rather than representing conflicting opinions, these responses are consistent with an ongoing technological transition in which AI-assisted development is progressively becoming part of software engineering practice.

Although deterministic automation remains the dominant approach for large-scale Type Hint modernization, the reported growing acceptance of Foundation Models suggests that modernization workflows are likely to become increasingly hybrid and perceived as valuable for supporting activities that require contextual understanding, planning or developer assistance.

7 Threats to Validation

Throughout this study, we consider that our methods were effective in supporting the analysis of refactoring practices related to Type Hint adoption and software modernization in Python projects. However, several limitations may influence the interpretation of the results.

In this work, we operationalize the concept of software rejuvenation as refactoring activities related to the adoption of Type Hints. Although this definition captures an important aspect of modernization in the Python ecosystem, it represents only a partial view of the broader concept of software rejuvenation, not covering other forms of code evolution and improvement.

In addition, the dataset is restricted to open-source Python projects hosted on GitHub, selected based on activity and maturity criteria. While this enables the analysis of relevant and actively maintained

systems, it may introduce a bias toward more established repositories, limiting generalization to smaller projects, other programming languages, or industrial contexts. The data selection process may also introduce sampling bias, since we focused on commits with a large number of detected transformations and selected one representative commit per developer, which may overrepresent large-scale modernization efforts and underrepresent incremental changes.

Regarding data collection, we relied on GitHub APIs to extract commits, pull requests, and associated discussions. Although this ensures reproducibility, platform limitations such as incomplete metadata and the absence of external communication channels may affect the full reconstruction of development context.

Since this study uses thematic analysis, there is an inherent risk of interpretative bias, as different researchers may code the same data in different ways. To mitigate this risk, we adopted independent coding, iterative refinement of the codebook, and consensus-based discussion, although some degree of subjectivity remains unavoidable due to the qualitative nature of the analysis.

Additionally, the survey introduces self-selection and response bias, as participation was voluntary and only 20 out of 117 contacted developers responded (17.09% response rate). This may limit the representativeness of the sample, as responses may be skewed toward developers more engaged with or interested in Type Hint modernization. Recall bias may also affect how developers report past decisions, potentially influencing response accuracy.

Finally, the results reflect a specific temporal window of the Python ecosystem. Given the rapid evolution of development tools, especially with the increasing adoption of LLM-based systems, the findings should be interpreted as representative of the observed period rather than a stable characterization of current or future practices.

8 Conclusion

This study investigated the motivations, decision-making process, and perceived role of Foundation Models in large-scale Type Hint modernization by combining repository mining, developer surveys, and qualitative thematic analysis. We found that these efforts are primarily driven by software maintainability, improvements to type information, Python language evolution, and compatibility requirements. The triangulation of repository artifacts and developer feedback revealed a high degree of consistency across data sources while also demonstrating their complementary nature: repository artifacts capture the technical aspects of modernization, while developers provide additional insight into the underlying rationale and decision-making process.

Our findings further show that large-scale Type Hint modernization is commonly supported by automation, with developers relying on specialized tools while reserving manual effort for validation and project decisions. Although Foundation Models were rarely used in the analyzed cases, most participants viewed them as promising assistants for repetitive modernization tasks, provided that human oversight is maintained. These results contribute to a better understanding of the factors driving large-scale Type Hint modernization and highlight the emerging interest in the use of Foundation Models to support these activities, motivating further research on AI-assisted modernization techniques.

Acknowledgements

<text included by the authors>

References

- [1] Robert L. Akers, Ira D. Baxter, Michael Mehlich, Brian J. Ellis, and Kenn R. Luecke. 2007. Case study: Re-engineering C++ component models via automatic program transformation. *Information and Software Technology* 49, 3 (2007), 275–291. doi:10.1016/j.infsof.2006.10.012 12th Working Conference on Reverse Engineering.
- [2] Eric Asare and Sarah Nadi. 2026. How do third-party Python libraries use type annotations?. In *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR)*. To appear, To appear, To appear. <https://sanadlab.org/pdf/AsareMSR2026.pdf> Accepted at ICSE 2026.
- [3] ATLAS.ti Scientific Software Development GmbH. 2025. ATLAS.ti 25. <https://atlasti.com> Qualitative Data Analysis Software.
- [4] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. arXiv:<https://doi.org/10.1191/1478088706qp0630a> doi:10.1191/1478088706qp0630a
- [5] Pedro V. R. de Carvalho, Rodrigo Bonifácio, Walter Lucas, and Alana Paula Barbosa Mota. 2024. Mining Discussions on Software Migration: A study of the Boost mailing list regarding C++ code evolution. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '24)*. Association for Computing Machinery, New York, NY, USA, 264–274. doi:10.1145/3701625.3701665
- [6] Wesley J. Chun. 2000. *Core Python Programming*. Prentice Hall PTR, Upper Saddle River, NJ.
- [7] Walter Lucas Monteiro de Mendonça. 2026. *A Socio-Technical Grounded Theory about Source Code Rejuvenation*. PhD thesis. Universidade de Brasília, Brasília, DF. Available at https://github.com/waltim/waltim.github.io/blob/master/tese_doutorado_walter_lucas__versao_final_.pdf.
- [8] Luca Di Grazia and Michael Pradel. 2022. The evolution of type annotations in python: an empirical study. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 209–220. doi:10.1145/3540250.3549114
- [9] Robert Dyer and Jigyasa Chauhan. 2022. An exploratory study on the predominant programming paradigms in Python code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022* (Singapore, Singapore), Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, Singapore, Singapore, 684–695. doi:10.1145/3540250.3549158
- [10] Rishi Bommasani et al. 2021. On the Opportunities and Risks of Foundation Models. *CoRR* abs/2108.07258 (2021). arXiv:2108.07258 <https://arxiv.org/abs/2108.07258>
- [11] Python Software Foundation. 2006. PEP 3107 – Function Annotations. Python Enhancement Proposal.
- [12] Python Software Foundation. 2015. PEP 484 – Type Hints. <https://peps.python.org/pep-0484/>. Python Enhancement Proposal, Status: Final, Python Version: 3.5, Accessed: 2026-04-16.
- [13] Python Software Foundation. 2016. PEP 526 – Syntax for Variable Annotations. Python Enhancement Proposal, Standards Track. Final. Topic: Typing. Created: 09-Aug-2016. Python-Version: 3.6. Post-History: 30-Aug-2016, 02-Sep-2016. Resolution: Python-Dev message..
- [14] Python Software Foundation. 2019. PEP 585 – Type Hinting Generics In Standard Collections. Python Enhancement Proposal, Standards Track. Final. Topic: Typing. Created: 03-Mar-2019. Python-Version: 3.9. Resolution: Python-Dev thread. Discussions-To: Typing-SIG list..
- [15] Walter Lucas, Fausto Carvalho, Rafael Campos Nunes, Rodrigo Bonifácio, João Saraiva, and Paola R. G. Accioly. 2024. Embracing modern C++ features: An empirical assessment on the KDE community. *J. Softw. Evol. Process.* 36, 5 (2024), e2605. doi:10.1002/SMR.2605
- [16] Walter Lucas, Rafael Campos Nunes, Rodrigo Bonifácio, Fausto Carvalho, Ricardo Lima, Michael Silva, Adriano Torres, Paola R. G. Accioly, Eduardo Monteiro, and João Saraiva. 2025. Understanding the adoption of modern Javascript features: An empirical study on open-source systems. *Empir. Softw. Eng.* 30, 3 (2025), 107. doi:10.1007/S10664-025-10663-9
- [17] Brian A. Malloy and James F. Power. 2019. An empirical analysis of the transition from Python 2 to Python 3. *Empirical Softw. Engg.* 24, 2 (April 2019), 751–778. doi:10.1007/s10664-018-9637-2
- [18] Lorelli S. Nowell, Jill M. Norris, Deborah E. White, and Nancy J. Moules. 2017. Thematic Analysis: Striving to Meet the Trustworthiness Criteria. *International Journal of Qualitative Methods* 16 (2017), 1–13. doi:10.1177/1609406917733847
- [19] Jeffrey L. Overbey and Ralph E. Johnson. 2009. Regrowing a language: refactoring tools allow programming languages to evolve. In *Proceedings of the 24th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2009, October 25-29, 2009, Orlando, Florida, USA* (New York, NY, USA), Shail Arora and Gary T. Leavens (Eds.). ACM, New York, NY,

- USA, 493–502. doi:10.1145/1640089.1640127
- [20] Peter Pirkelbauer, Damian Dechev, and Bjarne Stroustrup. 2010. Source Code Rejuvenation Is Not Refactoring. In *SOFSEM 2010: Theory and Practice of Computer Science, 36th Conference on Current Trends in Theory and Practice of Computer Science, Spindleruv Mlýn, Czech Republic, January 23-29, 2010. Proceedings (Lecture Notes in Computer Science)*, Jan van Leeuwen, Anca Muscholl, David Peleg, Jaroslav Pokorný, and Bernhard Rumpe (Eds.). Springer, Spindleruv Mlyn, Czech Republic, 639–650. doi:10.1007/978-3-642-11266-9_53
 - [21] Peter Pirkelbauer, Damian Dechev, and Bjarne Stroustrup. 2010. Source Code Rejuvenation Is Not Refactoring. In *SOFSEM 2010: Theory and Practice of Computer Science, 36th Conference on Current Trends in Theory and Practice of Computer Science, Spindleruv Mlýn, Czech Republic, January 23-29, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 5901)*, Jan van Leeuwen, Anca Muscholl, David Peleg, Jaroslav Pokorný, and Bernhard Rumpe (Eds.). Springer, Spindleruv Mlyn, Czech Republic, 639–650. doi:10.1007/978-3-642-11266-9_53
 - [22] Philippe Prados, Maggie Moss, Chris Angelico, and Guido van Rossum. 2021. PEP 604 – Allow writing union types as X | Y. Python Enhancement Proposal. <https://peps.python.org/pep-0604/>. Accessed: 2026-06-26.
 - [23] Ingkarat Rak-amnouykit, Daniel McCrevan, Ana Milanova, Martin Hirzel, and Julian Dolby. 2020. Python 3 types in the wild: a tale of two type systems. In *Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages (Virtual, USA) (DLS 2020)*. Association for Computing Machinery, New York, NY, USA, 57–70. doi:10.1145/3426422.3426981
 - [24] Charles Severance. 2015. Guido van Rossum: The Early Years of Python . *Computer* 48, 02 (Feb. 2015), 7–9. doi:10.1109/MC.2015.45
 - [25] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2022. RefactoringMiner 2.0. *IEEE Trans. Software Eng.* 48, 3 (2022), 930–950. doi:10.1109/TSE.2020.3007722
 - [26] Guido van Rossum. 2007. Python Programming Language. In *Proceedings of the 2007 USENIX Annual Technical Conference, USENIX ATC 2007, Santa Clara, CA, USA, June 17-22, 2007*, Jeff Chase and Srinivasan Seshan (Eds.). USENIX, USENIX ATC 2007, USA, –.